llm_enhanced_house_price_prediction_task120260126

January 26, 2026

[130]:
```
# Import necessary libraries
import pandas as pd
import numpy as np
import re
import os
import math
import json
from datetime import datetime
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import ftfy
```

0.1 Phase 1: Data Preparation & Understanding

[131]:
```
# Define file paths
# Get the current directory (where the notebook is located)
current_dir = os.path.dirname(os.path.abspath('__file__')) if '__file__' in␣
 ↪globals() else os.getcwd()

# Construct file paths
data_file = os.path.join(current_dir, 'data', 'realestate_data_london_2024_nov.
 ↪csv')
output_dir = os.path.join(current_dir, 'output')
output_file = os.path.join(output_dir, 'df_cleaned.csv')

# Create output directory if it doesn't exist
os.makedirs(output_dir, exist_ok=True)
```

0.1.1 1.1 Load and Explore the Dataset

[132]:
```
# Load the dataset
print("Loading dataset...")
try:
    df = pd.read_csv(data_file, encoding="utf-8")
    print(f" Dataset loaded successfully from: {data_file}")
except FileNotFoundError:
    print(f" Error: File not found at {data_file}")
```

```
    print("Current working directory:", os.getcwd())
    print("Available files in data directory:")
    data_dir = os.path.join(current_dir, 'data')
    if os.path.exists(data_dir):
        print(os.listdir(data_dir))
    raise
```

```
Loading dataset…
 Dataset loaded successfully from:
c:\Users\Admin\Python\S8_Thesis\llm\data\realestate_data_london_2024_nov.csv
```

[133]:
```
# Display initial dataset information
print(f"\n1. Dataset shape: {df.shape}")
print(f"   Rows: {df.shape[0]}, Columns: {df.shape[1]}")

print(f"\n2. Columns:")
for i, col in enumerate(df.columns.tolist(), 1):
    print(f"   {i:2d}. {col}")

print(f"\n3. Missing values check:")
missing_values = df.isnull().sum()
if missing_values.sum() > 0:
    print("   Missing values found:")
    for col, missing_count in missing_values[missing_values > 0].items():
        missing_percent = (missing_count / len(df)) * 100
        print(f"   - {col}: {missing_count} missing ({missing_percent:.2f}%)")
else:
    print("    No missing values found in any column")

print(f"\n4. Data types:")
print(df.dtypes)

print(f"\n5. First 3 rows of original data:")
print(df.head(3))
```

```
1. Dataset shape: (1019, 9)
   Rows: 1019, Columns: 9

2. Columns:
    1. addedOn
    2. title
    3. descriptionHtml
    4. propertyType
    5. sizeSqFeetMax
    6. bedrooms
    7. bathrooms
    8. listingUpdateReason
```

```
    9. price

3. Missing values check:
   Missing values found:
   - addedOn: 8 missing (0.79%)
   - sizeSqFeetMax: 150 missing (14.72%)
   - bedrooms: 16 missing (1.57%)
   - bathrooms: 35 missing (3.43%)

4. Data types:
addedOn                 object
title                   object
descriptionHtml         object
propertyType            object
sizeSqFeetMax          float64
bedrooms               float64
bathrooms              float64
listingUpdateReason     object
price                   object
dtype: object

5. First 3 rows of original data:
                addedOn                                              title  \
0            10/10/2024  8 bedroom house for sale in Winnington Road, H…
1  Reduced on 24/10/2024  7 bedroom house for sale in Brick Street, Mayf…
2  Reduced on 22/02/2024  6 bedroom terraced house for sale in Chester S…

                                  descriptionHtml propertyType  \
0  This magnificent home, set behind security gat…        House
1  In the heart of exclusive Mayfair, this majest…        House
2  A freehold home that gives you everything you …     Terraced

   sizeSqFeetMax  bedrooms  bathrooms listingUpdateReason        price
0        16749.0       8.0        8.0                 new  £24,950,000
1        12960.0       7.0        7.0       price_reduced  £29,500,000
2         6952.0       6.0        6.0       price_reduced  £25,000,000
```

0.1.2 1.2 Clean data

[134]:
```python
# Data Cleaning Functions
def clean_date(date_str):
    """
    Return '2024' for ALL rows, completely ignoring the original value
    """
    return '2024'  # Always return "2024" for all rows

def clean_description(html_text):
```

```
    """Clean description - remove HTML tags and extra whitespace"""
    # Handle missing values - return empty string instead of removing row
    if pd.isna(html_text):
        return ""

    # Convert to string
    text = str(html_text)

    # Remove HTML tags (preserve content between tags)
    text = re.sub(r'<[^>]+>', ' ', text)

    # Replace common HTML entities
    html_entities = {
        ' ': ' ',
        '&amp;': '&',
        '&lt;': '<',
        '&gt;': '>',
        '&quot;': '"',
        '&#39;': "'",
        '&rsquo;': "'",
        '&lsquo;': "'",
        '&rdquo;': '"',
        '&ldquo;': '"'
    }

    for entity, replacement in html_entities.items():
        text = text.replace(entity, replacement)

    # Clean up extra whitespace
    text = re.sub(r'\s+', ' ', text)

    # Strip leading/trailing whitespace
    return text.strip()

def clean_price(price_value):
    """Clean price - remove currency symbols and commas, convert to float"""
    # Handle missing values - return NaN but don't remove row
    if pd.isna(price_value):
        return np.nan

    # Convert to string
    price_str = str(price_value)

    # Extract all numbers (including decimals)
    # This preserves the numeric value regardless of format
    numbers = re.findall(r'[\d,\.]+', price_str)
```

```
    if not numbers:
        return np.nan

    # Take the first number found (should be the price)
    price_num = numbers[0]

    # Clean the number
    # Remove commas (thousands separators)
    price_num = price_num.replace(',', '')

    # Handle cases where dot might be decimal separator
    # If there's a dot and it's not the last character, assume it's decimal
    if '.' in price_num and price_num.rfind('.') < len(price_num) - 1:
        # Already has decimal point, keep as is
        pass
    else:
        # No valid decimal point found
        pass

    # Convert to float
    try:
        return float(price_num)
    except:
        # If conversion fails, try to handle special cases
        try:
            # Remove any remaining non-numeric characters
            clean_num = re.sub(r'[^\d\.]', '', price_str)
            return float(clean_num) if clean_num else np.nan
        except:
            return np.nan

# Handle encoding issue
REPL = {
    "â€˜": "'",
    "â€™": "'",
    "â€œ": '"',
    "â€": '"',
    "â€“": "-",
    "â€”": "-",
    "â€¦": "...",
    "Â": "",      # common stray
}

def fix_leftovers(s: str) -> str:
    if not isinstance(s, str):
        return s
    for k, v in REPL.items():
```

```
        s = s.replace(k, v)

    # If there are still stray "â" or "â€" fragments, remove them.
    # This is a "last step" cleanup.
    s = s.replace("â€", "")
    s = s.replace("â", "")

    # normalize spaces
    s = re.sub(r"\s+", " ", s).strip()
    return s
```

[135]:
```
# Apply data cleaning
# Create a copy for cleaning
df_cleaned = df.copy()

# 1 Clean and rename 'addedOn' column
print("\n1. Cleaning 'addedOn' column...")
df_cleaned['Date'] = df_cleaned['addedOn'].apply(clean_date)
df_cleaned = df_cleaned.drop('addedOn', axis=1)

# 2 Clean and rename 'descriptionHtml' column
print("\n2. Cleaning 'descriptionHtml' column...")
df_cleaned['listingDescription'] = df_cleaned['descriptionHtml'].
 ↪apply(clean_description)
df_cleaned = df_cleaned.drop('descriptionHtml', axis=1)

# Calculate some statistics about the cleaned descriptions
desc_lengths = df_cleaned['listingDescription'].apply(len)
print(f"    HTML tags removed")
print(f"   Average description length: {desc_lengths.mean():.0f} characters")
print(f"   Min length: {desc_lengths.min()} characters")
print(f"   Max length: {desc_lengths.max()} characters")

# 3 Clean 'price' column
print("\n3. Cleaning 'price' column...")
original_price_sample = df_cleaned['price'].head(3).tolist()
df_cleaned['price'] = df_cleaned['price'].apply(clean_price)

# Remove rows with invalid prices
original_rows = len(df_cleaned)
df_cleaned = df_cleaned.dropna(subset=['price'])
rows_removed = original_rows - len(df_cleaned)

print(f"   Currency symbols and commas removed")
print(f"   Rows with invalid prices removed: {rows_removed}")
print(f"   Price range: £{df_cleaned['price'].min():,.2f} to␣
 ↪£{df_cleaned['price'].max():,.2f}")
```

```
print(f"   Average price: £{df_cleaned['price'].mean():,.2f}")

# 4 Handle encoding issue
df_cleaned["title"] = df_cleaned["title"].astype(str).apply(fix_leftovers)
df_cleaned["listingDescription"] = df_cleaned["listingDescription"].astype(str).
 ↪apply(fix_leftovers)

# 5 Check other columns
print("\n4. Checking other columns...")

# Check for missing values in other columns
missing_after_clean = df_cleaned.isnull().sum()
if missing_after_clean.sum() > 0:
    print("   Missing values after cleaning:")
    for col, missing_count in missing_after_clean[missing_after_clean > 0].
 ↪items():
        print(f"   - {col}: {missing_count} missing")
else:
    print("    No missing values in cleaned data")
```

```
1. Cleaning 'addedOn' column…

2. Cleaning 'descriptionHtml' column…
    HTML tags removed
   Average description length: 1616 characters
   Min length: 106 characters
   Max length: 9210 characters

3. Cleaning 'price' column…
   Currency symbols and commas removed
   Rows with invalid prices removed: 1
   Price range: £315,000.00 to £80,000,000.00
   Average price: £11,298,546.61

4. Checking other columns…
   Missing values after cleaning:
   - sizeSqFeetMax: 149 missing
   - bedrooms: 15 missing
   - bathrooms: 34 missing
```

[136]:
```
# Display cleaned dataset information
print(f"\n1. Cleaned dataset shape: {df_cleaned.shape}")
print(f"   Rows: {df_cleaned.shape[0]}, Columns: {df_cleaned.shape[1]}")

print(f"\n2. Cleaned columns:")
for i, col in enumerate(df_cleaned.columns.tolist(), 1):
```

```
    print(f"   {i:2d}. {col} (dtype: {df_cleaned[col].dtype})")

print(f"\n3. Sample of cleaned data (first 2 rows):")
print(df_cleaned.head(2))

print(f"\n4. Data types summary:")
print(df_cleaned.dtypes)

print(f"\n5. Basic statistics for numeric columns:")
numeric_cols = df_cleaned.select_dtypes(include=[np.number]).columns
if len(numeric_cols) > 0:
    print(df_cleaned[numeric_cols].describe())
else:
    print("   No numeric columns found")
```

```
1. Cleaned dataset shape: (1018, 9)
   Rows: 1018, Columns: 9

2. Cleaned columns:
    1. title (dtype: object)
    2. propertyType (dtype: object)
    3. sizeSqFeetMax (dtype: float64)
    4. bedrooms (dtype: float64)
    5. bathrooms (dtype: float64)
    6. listingUpdateReason (dtype: object)
    7. price (dtype: float64)
    8. Date (dtype: object)
    9. listingDescription (dtype: object)

3. Sample of cleaned data (first 2 rows):
                                                title propertyType  \
0  8 bedroom house for sale in Winnington Road, H…        House
1  7 bedroom house for sale in Brick Street, Mayf…        House

   sizeSqFeetMax  bedrooms  bathrooms listingUpdateReason       price  Date  \
0        16749.0       8.0        8.0                 new  24950000.0  2024
1        12960.0       7.0        7.0       price_reduced  29500000.0  2024

                                listingDescription
0  This magnificent home, set behind security gat…
1  In the heart of exclusive Mayfair, this majest…

4. Data types summary:
title                   object
propertyType            object
sizeSqFeetMax          float64
bedrooms               float64
```

```
bathrooms              float64
listingUpdateReason     object
price                  float64
Date                    object
listingDescription      object
dtype: object

5. Basic statistics for numeric columns:
       sizeSqFeetMax     bedrooms   bathrooms         price
count     869.000000  1003.000000  984.000000  1.018000e+03
mean     5232.871116     5.111665    4.648374  1.129855e+07
std     11796.770144     3.264992    3.085809  6.940021e+06
min       425.000000     1.000000    1.000000  3.150000e+05
25%      2885.000000     3.000000    3.000000  7.500000e+06
50%      3834.000000     5.000000    4.000000  9.350000e+06
75%      5745.000000     6.000000    5.000000  1.300000e+07
max    336989.000000    66.000000   66.000000  8.000000e+07
```

0.1.3 1.3 Save cleaned data

[137]:
```python
# Save cleaned data
df_cleaned.to_csv(output_file, index=False)
print(f" Cleaned data saved to: {output_file}")
```

```
 Cleaned data saved to:
c:\Users\Admin\Python\S8_Thesis\llm\output\df_cleaned.csv
```

0.2 Phase 2: Use NuExtract-1.5 to extract features

0.2.1 2.1 Load NuExtract-1.5 (HF transformers) on CPU

[138]:
```python
# Load NuExtract-1.5 (HF transformers) on CPU
model_name = "numind/NuExtract-1.5"  # HF repo

print("Loading model with CPU+GPU offload...")
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,   # good for GPU+CPU offload
    device_map="auto",           # let HF/accelerate split across GPU and CPU
    trust_remote_code=True
).eval()

tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    trust_remote_code=True
)

print("Model and tokenizer loaded with device_map=auto.")
```

```
Loading model with CPU+GPU offload…

`flash-attention` package not found, consider installing for better performance:
No module named 'flash_attn'.
Current `flash-attention` does not support `window_size`. Either upgrade or use
`attn_implementation='eager'`.
Loading checkpoint shards: 100%|      | 2/2 [00:41<00:00, 20.61s/it]
Some parameters are on the meta device because they were offloaded to the disk
and cpu.

Model and tokenizer loaded with device_map=auto.
```

0.2.2 2.2 Define JSON template (schema)

[139]:
```python
# Define JSON template (schema)
template_dict = {
    "luxury_features": "",
    "transport_mentions": "",
    "school_mentions": "",
    "renovation_mentions": ""
}
template_str = json.dumps(template_dict, indent=4)

def build_prompt(text: str) -> str:
    instructions = """
You are extracting information from a real-estate listing.
Rules:
- Fill each field with exact phrases copied from the text.
- If the information is not mentioned, set the field to "".
- Do not paraphrase or invent information.
Fields:
- "luxury_features": phrases describing luxury amenities (pool, spa, gym,␣
 ↪cinema room, wine cellar, staff quarters, security, designer finishes, etc.).
- "transport_mentions": phrases describing transport links (stations,␣
 ↪underground lines, quick access to areas, etc.).
- "school_mentions": phrases describing schools (good schools, excellent␣
 ↪schools, named schools, etc.).
- "renovation_mentions": phrases indicating renovation or condition (newly␣
 ↪renovated, recently refurbished, turnkey, shell condition, etc.).
"""
    return f"""<|input|>
{instructions.strip()}

### Template:
{template_str}
### Text:
{text}
```

```
<|output|>"""
```

0.2.3 2.3 Function: run NuExtract on a batch of texts

[140]:
```python
# Function: run NuExtract on a batch of texts
def nuextract_batch(texts, batch_size=1, max_length=4096, max_new_tokens=160):
    prompts = [build_prompt(t) for t in texts]
    outputs = []

    with torch.no_grad():
        for i in range(0, len(prompts), batch_size):
            batch_prompts = prompts[i:i+batch_size]

            enc = tokenizer(
                batch_prompts,
                return_tensors="pt",
                truncation=True,
                padding=True,
                max_length=max_length
            )

            # IMPORTANT CHANGE: let HF/accelerate handle devices
            # If you really want to be explicit:
            # device = getattr(model, "device", torch.device("cpu"))
            # enc = {k: v.to(device) for k, v in enc.items()}

            pred_ids = model.generate(
                **enc,
                max_new_tokens=max_new_tokens,
                do_sample=False,
                use_cache=True
            )

            decoded = tokenizer.batch_decode(pred_ids, skip_special_tokens=True)

            for out in decoded:
                if "<|output|>" in out:
                    outputs.append(out.split("<|output|>")[1].strip())
                else:
                    outputs.append(out.strip())
    return outputs
```

0.2.4 2.4 Run extraction over df in small batches

[141]:
```
# Run extraction over df in small batches
df_sample = df_cleaned.iloc[:100, :]
texts = df_sample["listingDescription"].fillna("").astype(str).tolist()

batch_size = 4  # Increase this if GPU memory allows
print("Running NuExtract on", len(texts), "descriptions...")
raw_outputs = nuextract_batch(texts, batch_size=batch_size)
print("Got outputs:", len(raw_outputs))

for i, out in enumerate(raw_outputs[:3]):
    print(f"RAW {i}:", out)
```

```
Running NuExtract on 100 descriptions…
Got outputs: 100
RAW 0: {"luxury_features": "luxurious and expansive entertaining areas, state-
of-the-art gym and spa with an indoor swimming pool, sauna, steam room, and pool
lounge area, games room, media room, and a beautifully landscaped private rear
garden", "transport_mentions": "quick access to the City and the West End",
"school_mentions": "excellent schools", "renovation_mentions": ""}
RAW 1: {"luxury_features": "swimming pool, cinema room and separate staff
accommodation", "transport_mentions": "just off Park Lane and a stone's throw
away from Hyde Park and Green Park", "school_mentions": "",
"renovation_mentions": ""}
RAW 2: {"luxury_features": "", "transport_mentions": "", "school_mentions": "",
"renovation_mentions": ""}
```

[142]:
```
# Save raw_outputs to JSONL
output_path = "raw_outputs.jsonl"

with open(output_path, "w", encoding="utf-8") as f:
    for out in raw_outputs:
        f.write(out.strip() + "\n")

print("Saved raw_outputs to:", output_path)
```

```
Saved raw_outputs to: raw_outputs.jsonl
```

0.2.5 2.5 Parse JSON and attach to your DataFrame

[143]:
```
parsed = []
for out in raw_outputs:
    try:
        obj = json.loads(out)
    except json.JSONDecodeError:
        # basic fallback: try to isolate JSON if extra text appears
        start = out.find("{")
        end = out.rfind("}")
```

```
        if start != -1 and end != -1 and end > start:
            obj = json.loads(out[start:end+1])
        else:
            obj = {
                "luxury_features": "",
                "transport_mentions": "",
                "school_mentions": "",
                "renovation_mentions": ""
            }
    parsed.append(obj)

df_extracted = pd.DataFrame(parsed)
df_final = pd.concat([df_sample.reset_index(drop=True), df_extracted], axis=1)

# save df_final to output folder
output_dir = os.path.join(current_dir, "output")
os.makedirs(output_dir, exist_ok=True)

final_file = os.path.join(output_dir, "df_final_with_extracted_features.csv")
df_final.to_csv(final_file, index=False)
print(f"df_final saved to: {final_file}")
```

```
df_final saved to:
c:\Users\Admin\Python\S8_Thesis\llm\output\df_final_with_extracted_features.csv
```